# Automata Theory

## Decidability

CS32022

**By Thisara Watawana**

06 October 2026

# 8. Decidability

## The Search for a Definition of "Algorithm"

- **Hilbert's Challenge (1900):** At the International Congress of Mathematicians, David Hilbert presented 23 fundamental problems for the 20th century.
- **Problem #10 (Polynomial Diophantine Equations):** Devise a "general mechanical procedure" to determine whether an arbitrary polynomial equation with integer coefficients has integer roots.

$$\textbf{Example: } 3x^2 - 2xy - z^7 = 0 \quad \Longleftrightarrow ? \Longrightarrow \quad x, y, z \in \mathbb{Z}$$

- **The Mathematical Obstacle:**
  - To show a problem can be solved, you only need to invent one specific working algorithm.
  - To show a problem cannot be solved, you must prove that no possible algorithm could ever exist.
- **The Core Gap:** In 1900, mathematics lacked a precise, formal definition of what an "algorithm" actually was.

# 8. Decidability

## The Church-Turing Thesis

- **The Breakthrough (1936):** Two independent mathematical frameworks were published to formally define computable functions:
    - **Alonzo Church (λ-Calculus):** Formalized computation using functional abstraction, variable binding, and substitution rules $(\lambda x.M)N$.
    - **Alan Turing (Turing Machines):** Formalized computation using a mechanical model mimicking a human mathematician reading/writing on paper tape.
    - **The Equivalence Theorem:** Church and Turing proved that λ-Calculus and Turing Machines are computationally equivalent. Any function computable by one model is computable by the other.
- **Core Definition:** The intuitive, informal concept of an algorithm (or effective procedure) is identical to the formal mathematical concept of a Turing Machine that halts on all inputs (a Decider).

**Intuitive Concept of an Algorithm ⇔ Turing Machine Decider**

# 8. Decidability

## Robustness and the Universality of Computation

- **Hardware Independence:** Adding fancy features (multiple tapes, random access memory pointers, non-determinism, quantum superpositions) changes execution speed, but never changes the set of solvable problems.
- **The Universal Boundary:**

  Python / C++ / Java Code ≡ Supercomputers ≡ 1-Tape Turing Machine

- **The Direct Consequence for Decidability:**
  - If a problem can be solved by a Turing Machine ⇒ It is algorithmic / decidable.
  - If a problem cannot be solved by a Turing Machine ⇒ It is fundamentally impossible for ANY algorithm or physical computer to solve.

# 8. Decidability

## The Halting Problem ($HALT_{TM}$)

**The Problem Statement:** Can we build a universal program analyzer that inspects any given computer program and its input, and determines whether that program will eventually finish running or get stuck in an infinite loop?

**Formal Language Definition:**

$HALT_{TM} = \{ \langle M, w \rangle \mid M \text{ is a Turing Machine and } M \text{ halts on input } w \}$

**The Target Decider ($H$):** We want a decider machine $H$ that takes an encoding $\langle M, w \rangle$ and behaves as follows:

$H(\langle M, w \rangle) =$ **ACCEPT** if $M$ halts on input $w$

$H(\langle M, w \rangle) =$ **REJECT** if $M$ loops forever on input $w$

**Core Goal:** Prove whether such a decider $H$ can exist as a valid algorithm.

# 8. Decidability

## The Halting Problem: The Proof

**Step 1 (Hypothesis):** Assume that $HALT_{TM}$ is decidable. Therefore, a Turing Machine decider $H$ exists that always halts and correctly identifies whether $M(w)$ halts or loops.

**Step 2 (Constructing Machine *D*):** Use $H$ as a subroutine to build a new, malicious Turing Machine $D$.

**Input to *D*:** A description of a Turing Machine $\langle M \rangle$.

**Internal Behavior of *D*:**

- $D$ runs $H$ on input $\langle M, \langle M \rangle \rangle$ (testing what machine $M$ does when fed its own source code as input).
- $D$ does the exact opposite of whatever $H$ outputs:
  - If $H$ outputs **ACCEPT** ($M$ halts on $\langle M \rangle$) $\Rightarrow$ $D$ intentionally enters an infinite loop.
  - If $H$ outputs **REJECT** ($M$ loops on $\langle M \rangle$) $\Rightarrow$ $D$ halts immediately and accepts.

# 8. Decidability

## The Halting Problem: The Proof

**Step 3:** Now run Machine $D$ with its own source code $\langle D \rangle$ as input: $\boldsymbol{D(\langle D \rangle)}$

**Case 1: Suppose $D(\langle D \rangle)$ HALTS.**

- $H(\langle D, \langle D \rangle \rangle)$ must output **ACCEPT** (because $H$ is a decider that predicts $D$ halts).
- By $D$'s internal construction, when $H$ outputs **ACCEPT**, $D$ enters an infinite loop.
- **Result:** $D(\langle D \rangle)$ LOOPS FOREVER $\Rightarrow$ **Contradiction!**

**Case 2: Suppose $D(\langle D \rangle)$ LOOPS FOREVER.**

- $H(\langle D, \langle D \rangle \rangle)$ must output **REJECT** (because $H$ predicts $D$ loops).
- By $D$'s internal construction, when $H$ outputs **REJECT**, $D$ halts immediately.
- **Result:** $D(\langle D \rangle)$ HALTS $\Rightarrow$ **Contradiction!**

**Summary: $D(\langle D \rangle)$ Halts $\Leftrightarrow$ $D(\langle D \rangle)$ Loops Forever**

# 8. Decidability

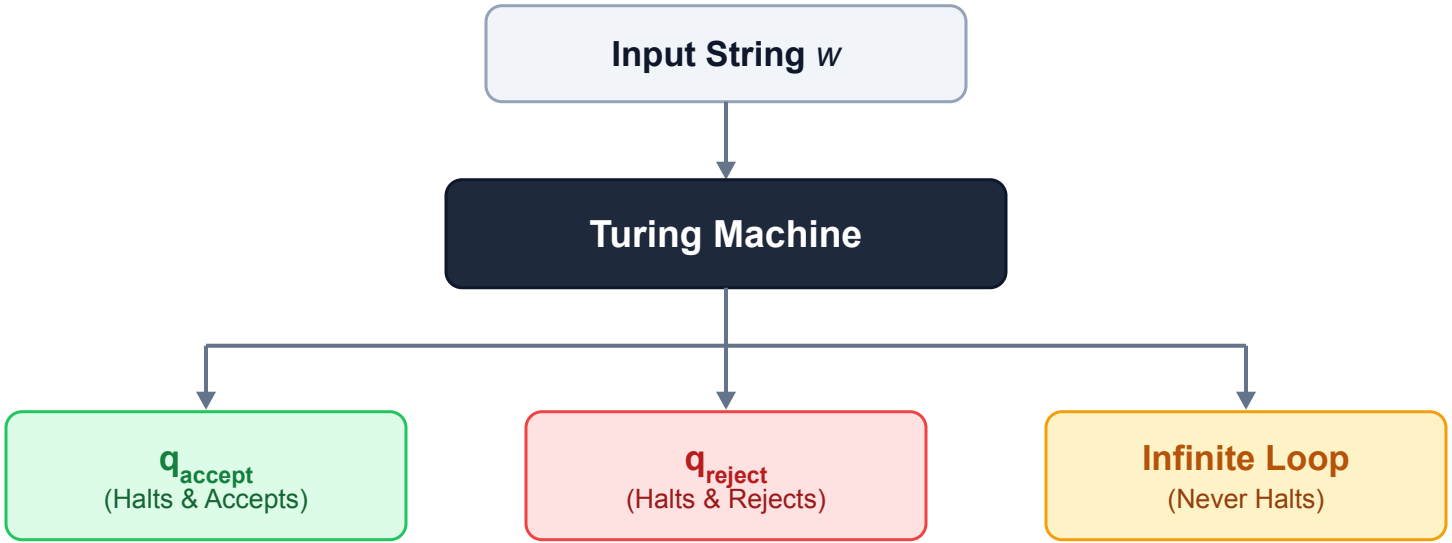
## Turing's Undecidability Theorem

- **Theorem (Alan Turing, 1936):** $\text{HALT}_{\text{TM}}$ is undecidable. No general algorithm exists to determine whether an arbitrary program halts on a given input.
- **Conclusion of the Proof:** The logical paradox $D(\langle D \rangle) \Leftrightarrow \neg D(\langle D \rangle)$ proves that our initial assumption was false. The decider machine $H$ cannot exist.
- **Takeaway for Computer Science:**
  - **No Perfect Static Analyzers:** It is mathematically impossible to write a program that checks all other programs for infinite loops, deadlocks, or runtime exceptions.
  - **The Ultimate Ceiling:** Unlike the Pumping Lemma for Finite Automata (which showed $a^n b^n$ just needed a stack), hitting $\text{HALT}_{\text{TM}}$ proves that no computing device, past, present, or future, can overcome this boundary.

# 8. Decidability

## Decidable (*R*) vs. Recognizable (*RE*) Languages

Having established that $HALT_{TM}$ hits a computational wall, we formally categorize languages based on how Turing Machines handle them:



# 8. Decidability

## Decidable (*R*) vs. Recognizable (*RE*) Languages

### 1. Turing-Decidable Languages (*R*) "Recursive"

**Definition:**

A language $L$ is decidable if a Turing Machine $M$ exists such that:

$w \equiv w \in L \Rightarrow M$ halts in $q_{accept}$

$w \equiv w \notin L \Rightarrow M$ halts in $q_{reject}$

**Key Feature:**

$M$ is a Decider - infinite loops are strictly prohibited for all inputs.

### 2. Turing-Recognizable Languages (*RE*) "Recursively Enumerable"

**Definition:**

A language $L$ is recognizable if a Turing Machine $M$ exists such that:

$w \in L \Rightarrow M$ halts in $q_{accept}$

$w \notin L \Rightarrow M$ halts in $q_{reject}$ **OR loops forever (∞)**

**Key Feature:**

Acceptance is guaranteed in finite time, but rejection may hang indefinitely.

# 8. Decidability

## Proving R ⊊ RE (Decidable is a Strict Subset)

**Is $HALT_{TM}$ Turing-Recognizable? YES.**

**Recognizer Construction (S):** Given input ⟨M, w⟩, simulator S simply runs M on w.

If M halts ⇒ S accepts. If M loops ⇒ S loops.

- **Thus, $HALT_{TM} \in RE$.**

**Is $HALT_{TM}$ Turing-Decidable? NO.**

Turing's proof demonstrated that no decider can avoid infinite loops for all pairs ⟨M, w⟩.

- **Thus, $HALT_{TM} \notin R$.**

**Conclusion: $HALT_{TM} \in RE \setminus R \Rightarrow R \subsetneq RE$**

# 8. Decidability

## The Complement Theorem

**Theorem:** A language $L$ is decidable if and only if $L$ is Turing-recognizable ($L \in$ **RE**) AND its complement $\overline{L}$ is Turing-recognizable ($\overline{L} \in$ **RE**).

$$\textbf{Decidable } (\boldsymbol{R}) = \boldsymbol{RE} \cap \textbf{co-}\boldsymbol{RE}$$

**Proof Intuition:**

Run the recognizer for $L$ ($M_1$) and the recognizer for $\overline{L}$ ($M_2$) in parallel on input $w$. Since $w$ must belong to either $L$ or $\overline{L}$, exactly one machine is guaranteed to accept and halt. Thus, the parallel combination never loops forever.

# 8. Decidability

## Beyond Recognizability ($R\bar{E}$)

### The Complement Theorem

**Theorem:** A language $L$ is decidable if and only if $L$ is Turing-recognizable ($L \in$ **RE**) AND its complement $\bar{L}$ is Turing-recognizable ($\bar{L} \in$ **RE**).

$$\textbf{\textit{Decidable (R)}} = \textbf{\textit{RE}} \cap \textbf{co-}\textbf{\textit{RE}}$$

- **Proof Intuition:** Run recognizer $M_1$ for $L$ and recognizer $M_2$ for $\bar{L}$ in parallel on input $w$. Since $w$ belongs to either $L$ or $\bar{L}$, exactly one machine accepts and halts. Thus, the combination never loops forever.
- **The Unrecognizable Boundary ($HALT\bar{}_{TM}$)**
  - We know $\textbf{\textit{HALT}}_{TM} \in$ **RE** (it is Turing-recognizable).
  - We know $\textbf{\textit{HALT}}_{TM} \notin$ **R** (it is undecidable).
  - By the Complement Theorem, the complement $\textbf{\textit{HALT}}\bar{}\square\square$ (machines that loop forever) cannot be Turing-recognizable:

$$\textbf{\textit{HALT}}\bar{}_{TM} \notin \textbf{RE}$$

# 8. Decidability

## The Master Hierarchy of Computability

**ALL LANGUAGES / PROBLEMS** Uncountable Set: $2^{\aleph_0}$

**TURING-RECOGNIZABLE / RE** Countable Set: $\aleph_0$

**RE \ R Examples:** $HALT_{TM}$ , $A_{TM}$ $HALT_{\varepsilon}$

**TURING-DECIDABLE / R** Countable Set: $\aleph_0$

**Decidable Problem Examples:**

$A_{DFA}$ $E_{DFA}$ $EQ_{DFA}$ $A_{CFG}$ $E_{CFG}$

**UNRECOGNIZABLE / NON-RE** (Outside RE Boundary)

**Examples:**

Complement $HALT_{TM}$

$\bar{A}_{TM}$

# 8. Decidability

## Countable vs. Uncountable Sets

**Definition of Countable:** A set $S$ is countably infinite if its elements can be put into a one-to-one correspondence (1-to-1) with the set of natural numbers $\mathbb{N} = \{1, 2, 3, \ldots\}$.

$$\text{An explicit listing exists: } S = \{s_1, s_2, s_3, s_4, \ldots\}$$

### Fact 1: The Set of All Strings ($\Sigma^*$) is Countable

We can list all strings in lexicographic order:

$$\Sigma^* = \{\varepsilon, 0, 1, 00, 01, 10, 11, 000, \ldots\}$$

### Fact 2: The Set of All Turing Machines is Countable

Every Turing Machine $M$ can be encoded as a finite string of bytes $\langle M \rangle$. Sorting these byte strings lexicographically gives a complete, numbered list of all possible programs:

$$\text{All TMs} = \{M_1, M_2, M_3, M_4, \ldots\}$$

# 8. Decidability

## Cantor's Diagonalization Method (1891)

**Goal:** Prove that the set of all possible languages (power set $\mathscr{P}(\Sigma^*)$) cannot be listed in a countable sequence.

**The Infinite Binary Matrix:** Construct a grid where row $i$ represents Turing Machine $M_i$, column $j$ represents string $s_j$, and cell $(i, j)$ is 1 if $M_i$ accepts $s_j$, and 0 if it rejects/loops:

| | $s_1$ | $s_2$ | $s_3$ | $s_4$ | … |
|---|---|---|---|---|---|
| **$M_1$** | **1** | 0 | 1 | 0 | … |
| **$M_2$** | 1 | **1** | 0 | 0 | … |
| **$M_3$** | 0 | 1 | **0** | 1 | … |
| **$M_4$** | 1 | 0 | 1 | **1** | … |

# 8. Decidability

## Uncomputable Languages Outnumber Computable Ones

**Constructing the Diagonal Language ($L_d$):** Define a new language Ld by flipping every bit along the main diagonal:

$$\mathbf{s_i \in L_d \Leftrightarrow M_i \text{ does NOT accept } s_i}$$

$L_d$'s row pattern: **0**, **0**, **1**, **0**, …

**The Contradiction:** Is there any Turing Machine $M_k$ in our complete list that recognizes $L_d$?

Test $M_k$ on string $s_k$:

- If $M_k$ accepts $s_k$, then by definition of $L_d$, $s_k \notin L_d$.
- If $M_k$ does not accept $s_k$, then by definition of $L_d$, $s_k \in L_d$.

$M_k$ differs from $L_d$ at string $s_k$ for every single k.

# 8. Decidability

## Uncomputable Languages Outnumber Computable Ones

**Mathematical Conclusion:**

$|\text{Turing Machines}| = \aleph_0$ (Countably Infinite)

$|\text{All Languages}| = 2^{\aleph_0}$ (Uncountably Infinite)

**Turing Machines ($\aleph_0$) $\ll$ Languages ($2^{\aleph_0}$)**

# 8. Decidability

## Reducibility & The Limits of Computation

- Reducibility is a way of converting one problem into another problem.
- If problem *A* reduces to problem *B*, it means: "If I have a magical tool to solve *B*, I can use it to solve *A*."
- Example:
  - Problem *A*: Finding your way around a new city.
  - Problem *B*: Reading a map.
  - Reduction: If you can solve *B* (read a map), you automatically solve *A*. Therefore, *A* is no harder than *B*.
- Using Reductions to Prove Impossibility:
  - If *A* reduces to *B*, and we already know *A* is impossible to solve, what does that say about *B*?
  - Conclusion: *B* must also be impossible to solve
  - Logic: If *B* were solvable, we could use it to solve *A*. But mathematics has proven *A* is unsolvable. Therefore, the solver for *B* cannot exist.

# 8. Decidability

## The Direction of Reduction

**Notation:** A ≤□ B (Problem A mapping-reduces to Problem B)

To prove a New Problem (B) is undecidable, we must reduce from a Known Undecidable Problem (A) to the New Problem (B).

**The Correct Formula:**

- **Known Undecidable ≤□ New Unknown Problem**

**Assume:** Start by assuming the target problem B is decidable by some Turing Machine R.

**Construct:** Build a wrapper Turing Machine S to solve the known undecidable problem A (e.g., the Halting Problem).

**Wrap:** Inside S, format the input and feed it into the hypothetical decider R.

- **Contradict:** Show that S works perfectly. Because A is proven undecidable, S cannot exist. Thus, our assumption was false: R cannot exist, and B is undecidable.

# 8. Decidability

## The Direction of Reduction

**Notation:** A ≤□ B (Problem A mapping-reduces to Problem B)
To prove a New Problem (B) is undecidable, we must reduce from a Known Undecidable Problem (A) to the New Problem (B).

**The Correct Formula:**

- **Known Undecidable ≤□ New Unknown Problem**

**Assume:** Start by assuming the target problem B is decidable by some Turing Machine R.

**Construct:** Build a wrapper Turing Machine S to solve the known undecidable problem A (e.g., the Halting Problem).
**Wrap:** Inside S, format the input and feed it into the hypothetical decider R.

- **Contradict:** Show that S works perfectly. Because A is proven undecidable, S cannot exist. Thus, our assumption was false: R cannot exist, and B is undecidable.

# 8. Decidability

## Linking to the Halting Problem

### Our Anchor for Reductions

The Halting Problem (HALT□□):

**HALT□□ = { ⟨M, w⟩ | TM M halts on input w }**

- We know HALT□□ is undecidable (Turing, 1936). We will now use HALT□□ as our "Problem A" to prove three highly practical computer science problems are also undecidable.

# 8. Decidability

## Exercise 1 - The Blank Tape Problem

Can we write a static analysis tool that tells us if a program will eventually halt if we give it absolutely no input (a completely blank tape)?

**$HALT_\varepsilon$ = { ⟨M⟩ | M halts when run on a blank tape }**

**Exercise:** Using reducibility ($HALT_{TM} \leq_m HALT_\varepsilon$), prove that $HALT_\varepsilon$ is undecidable.

# 8. Decidability

## Exercise 1 - Solution

### Proof by Reduction:

1. Assume a decider R exists that solves $HALT_{\varepsilon}$.
2. Construct a machine S to solve HALT□□ on input ⟨M, w⟩:
   - Inside S, create the code for a new machine $M_w$:
     - Machine $M_w$: Ignore the input tape. Overwrite it with string w. Run M on w.
   - Pass $\langle M_w \rangle$ into our hypothetical decider R.
   - If R accepts, S accepts. If R rejects, S rejects.
3. Contradiction: $M_w$ halts on a blank tape if and only if M halts on w. We just used R to solve the general Halting Problem!
4. Since HALT□□ is undecidable, R cannot exist. Therefore, $HALT_{\varepsilon}$ is undecidable.

# 8. Decidability

## Exercise 2 - Dead Code Detection

In compiler optimization, we want to know if a specific line of code (state q) is "dead code" (never reached).

**STATE□□ = { ⟨M, w, q⟩ | M enters state q when run on w }**

**Exercise:** Prove that STATE□□ is undecidable by reducing HALT□□ to it.

# 8. Decidability

## Exercise 2 - Solution

**Proof by Reduction:**

1. Assume a decider R exists for STATE□□.
2. Construct a machine S to solve HALT□□ on input ⟨M, w⟩:
   - Inside S, modify M to create M':
     - Add a brand new state called $q_{trap}$.
     - Redirect all halting transitions (accept, reject, and blank cells) to point to $q_{trap}$.
   - Run the decider R on ⟨M', w, $q_{trap}$⟩.
   - If R says YES, S accepts. If R says NO, S rejects.
3. Contradiction: M' reaches q□$_{ra}$□ if and only if original M halts. S just solved the Halting Problem.
4. Therefore, STATE□□ is undecidable. (Perfect dead-code elimination is impossible!)

# 8. Decidability

## Exercise 3 - The Emptiness Problem

Can we write an algorithm to determine if a Turing Machine rejects everything? (i.e., its language is completely empty).

**E□□ = { ⟨M⟩ | L(M) = ∅ }**

**Exercise:** Prove E□□ is undecidable by reducing the Acceptance Problem (A□□) to E□□.

# 8. Decidability

## Exercise 3 - Solution

**Proof by Reduction:**

1. Assume a decider R exists for E□□.

2. Construct a machine S to solve A□□ on input ⟨M, w⟩:

   • Inside S, build a new machine $M_1$:

   - Machine $M_1$ (on input x): Ignore x. Simulate original M on string w. If M accepts w, Accept x.

   • Run R on ⟨$M_1$⟩.

   • Notice the flip: If R accepts (says L($M_1$) is empty), M did not accept w, so S should Reject. If R rejects, S should Accept.

3. Contradiction: S successfully decides A□□. But A□□ is undecidable.

4. Therefore, E□□ is undecidable.

# 8. Decidability

## Rice's Theorem

Any non-trivial semantic property of the language recognized by a Turing machine is undecidable.

**The Two Criteria:**

- **Semantic Property:** It is about the language L(M) (what the code accepts), not the syntax/code of M (e.g., how many states it has, or if it executes line 4).
- **Non-trivial:** At least one TM's language has the property, and at least one TM's language does not.

**Why it matters for students:**

- "Does M accept the string 'hello'?" ⇒ Undecidable by Rice's Theorem.
- "Does M accept a regular language?" ⇒ Undecidable by Rice's Theorem.
- "Does M accept an empty language (E□□)?" ⇒ Undecidable by Rice's Theorem.

# 8. Decidability

## Formal Mapping Reducibility ($A \leq_m B$) & Recognizability

**The Formal Definition:** $A \leq_m B$ if there exists a computable function $f: \Sigma^* \to \Sigma^*$ such that for every string $w$:

$$w \in A \Leftrightarrow f(w) \in B$$

**The Four Decision & Recognition Rules (Standard Multiple-Choice / True-False Material):**

- If $A \leq_m B$ and B is decidable, then A is decidable.
- If $A \leq_m B$ and A is undecidable, then B is undecidable.
- If $A \leq_m B$ and B is Turing-recognizable, then A is Turing-recognizable.
- If $A \leq_m B$ and A is NOT Turing-recognizable, then B is NOT Turing-recognizable.

# Questions?

thisaraw@boffin.lk